

Spiderman's Workout

Day 3 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“starts and finishes at street level”*
- ▶ *“minimizes the required building height”*

Programming interpretation:

- ▶ Each distance must be assigned either U or D in the original order.
- ▶ The path may never go below height zero and must end at zero.
- ▶ Among valid paths, minimize the highest height ever reached.

Textbook connection: CP4 3.5 f. DP Level 1

Core technique: DP over step index and current height

- ▶ For each distance, Spiderman chooses U or D.
- ▶ Height can never become negative.
- ▶ He must finish at height 0 while minimizing the highest height reached.

- ▶ A greedy choice can trap the path later.
- ▶ State must include how many moves are done and the current height.
- ▶ Path reconstruction is required to print U/D moves.

- ▶ Let `max_height` be the sum of all distances.
- ▶ `dp[i][h]` stores the smallest maximum height after `i` moves ending at `h`.
- ▶ From each reachable state, try moving up and down.
- ▶ Update `parent[i+1][new_h]` with previous height and move letter.
- ▶ If `dp[m][0]` is reachable, follow parent pointers backward to build the path.

- ▶ Moving up may increase the maximum height reached.
- ▶ Moving down preserves the previous maximum height.
- ▶ The parent table turns an optimal value DP into an output-producing DP.

- ▶ The DP considers both legal choices after every reachable prefix.
- ▶ For each state, it keeps the path with the smallest maximum height, which dominates worse paths to the same state.
- ▶ The final state (all moves, height 0) is exactly the problem's ending condition.

Complexity: $O(MS)$ time and $O(MS)$ memory, where S is the sum of distances.

- ▶ Return IMPOSSIBLE when height 0 is unreachable after all moves.
- ▶ The reconstructed path is built backward and then reversed.

Source lines: spiderman.py:44-66

```
import sys

def solve(distances):
    m = len(distances)

    max_height = sum(distances)

    INF = float("inf")

    dp = [[INF] * (max_height + 1) for _ in range(m + 1)]
    parent = [[None] * (max_height + 1) for _ in range(m + 1)]
    dp[0][0] = 0
```

What this chunk does

- ▶ The maximum possible height is the sum of all distances.
- ▶ $dp[i][h]$ is the lowest peak height for paths ending at h after i moves.
- ▶ `parent` stores the previous height and move so the final string can be rebuilt.

Source lines: spiderman.py:68-97

```
for i in range(m):
    distance = distances[i]

    for height in range(max_height + 1):

        if dp[i][height] == INF:
            continue

        new_height = height + distance

        if new_height <= max_height:

            highest = max(dp[i][height], new_height)

            if highest < dp[i + 1][new_height]:
                dp[i + 1][new_height] = highest

                parent[i + 1][new_height] = (height, "U")
```

What this chunk does

- ▶ Only reachable prior heights are expanded.
- ▶ Moving up changes the current height and may increase the path's maximum height.
- ▶ The transition is kept only if it improves the best known peak for the new state.

Source lines: `spiderman.py:98-113`

```
new_height = height - distance

if new_height >= 0:
    highest = dp[i][height]

    if highest < dp[i + 1][new_height]:
        dp[i + 1][new_height] = highest

    parent[i + 1][new_height] = (height, "D")
```

What this chunk does

- ▶ A downward move is legal only if the new height stays nonnegative.
- ▶ Moving down cannot increase the maximum height reached so far.
- ▶ The parent pointer records that this state came from a D move.

Source lines: spiderman.py:115-139

```
if dp[m][0] == INF:
    return "IMPOSSIBLE"

path = []
height = 0

for i in range(m, 0, -1):
    previous_height, move = parent[i][height]

    path.append(move)

    height = previous_height

path.reverse()

return "".join(path)
```

What this chunk does

- ▶ The only acceptable final height is zero.
- ▶ If `dp[m][0]` is unreachable, the answer is IMPOSSIBLE.
- ▶ Otherwise parent pointers rebuild the moves backward, then reverse them.

Source lines: spiderman.py:142-155

```
def main():  
    input = sys.stdin.buffer.readline  
  
    test_cases = int(input())  
  
    for _ in range(test_cases):  
        m = int(input())  
  
        distances = list(map(int, input().split()))  
  
        print(solve(distances))  
  
main()
```

What this chunk does

- ▶ Each test case provides the number of moves and the distance sequence.
- ▶ The solver returns either a U/D string or IMPOSSIBLE.

Solution file: `practice_contests/day_3/spiderman.py`

Problem: <https://open.kattis.com/problems/spiderman>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.